1 Growing Double-Gyre Flow and Trajectory Generation

This section implements a time-dependent switching double-gyre flow together with the numerical generation of particle trajectories on the domain $[0, 3] \times [0, 2]$, $t \in [0, 1]$.

The construction follows the growing coherent structure framework described in Section 6.1 of the paper *An Inflated Dynamic Laplacian to Track the Emergence and Disappearance of Coherent Sets*.

1.1 Velocity Field Definition

The file `GrowGyre.py` defines the non-autonomous velocity field

$$\dot{x} = v(t, x),$$

where the flow transitions between different gyre configurations through a time-dependent parameter $r(t)$.

Three transition regimes are implemented:

- **uniform**
$$r(t) = t,$$
producing a linear transition,
- **abrupt**
$$r(t) = \frac{1}{2}\left(1 + \tanh(10(t - 0.5))\right),$$
- **very-abrupt**
$$r(t) = \frac{1}{2}\left(1 + \tanh(40(t - 0.5))\right),$$
which creates a sharp switching behaviour near $t = 0.5$.

The separatrix deformation parameters are defined as

$$\alpha = \frac{\pi(1 - 2r)}{3(r - 2)(r + 1)},$$

and

$$\beta = \frac{2\pi - 9a}{3}.$$

For a particle position

$$x = (x_1, x_2),$$

the auxiliary quantities are

$$s_1 = \alpha x_1^2 + \beta x_1, \qquad s_2 = \frac{\pi}{2} x_2.$$

The velocity components are then computed as

$$u(x,t) = -\frac{\pi}{2}\cos(s_2)\sin(s_1),$$

and

$$v(x,t) = \sin(s_2)\cos(s_1)(2\alpha x_1 + \beta).$$

The function returns the velocity vector

$$(u, v).$$

The commented block in the code shows an equivalent formulation derived from the stream function

$$\psi(x,t) = -\sin\Big(\frac{\pi x_2}{2}\Big)\sin\big(\pi f(x_1,t)\big),$$

with

$$f(x_1,t) = \alpha x_1^2 + \beta x_1.$$

1.2 Particle Trajectory Generation

The file `generate_pts_GrowGyre.py` generates trajectories by integrating the velocity field over time.

A uniform grid of initial conditions is created using

$$mx = 45, \qquad my = 30,$$

giving

$$N = mx \cdot my = 1350$$

particles.

The spatial domain is

$$I_x = [0, 3], \qquad I_y = [0, 2].$$

Time integration is performed on

$$[0, 1]$$

with timestep

$$\Delta t = 0.01.$$

The resulting discrete time vector is

$$T_{\text{span}} = \{0, 0.01, 0.02, \ldots, 1\}.$$

For each initial condition, trajectories are computed by solving

$$\frac{dX}{dt} = 20\, v(t, X),$$

where the scalar factor 20 controls the flow strength.

The numerical integration uses `solve_ivp` with the adaptive Runge–Kutta method RK45, together with tolerances

$$\text{rtol} = 10^{-9}, \qquad \text{atol} = 10^{-12}.$$

The trajectory data are stored in the array

$$\text{pts} \in \mathbb{R}^{2\times N\times T},$$

where:

- the first dimension contains the spatial coordinates,
- the second dimension indexes particles,
- the third dimension indexes time.

To avoid recomputation, the trajectories are cached in a MATLAB `.mat` file containing:

- particle trajectories `pts`,
- time vector `Tspan`,
- grid parameters,
- flow strength,
- selected regime.

The commented plotting section visualizes the time evolution of particles by updating their positions dynamically, with colors assigned according to the initial x-coordinate.

2 Inflated Dynamic Laplacian Utilities and Parameter Selection

This section contains the utility imports together with the heuristic procedures used to determine the temporal diffusion parameter a and the kernel bandwidth parameter ε.

2.1 Package Imports

The initialization file imports the main modules required for the computation:

- `InflatedDynamicLaplacian` — main solver,
- `seba` — sparse eigenbasis approximation,

- diffusion maps utilities,
- spacetime diffusion matrix construction,
- plotting and visualization routines.

The plotting functions provide visualizations for:

- stream functions,
- eigenvalues,
- spacetime embeddings,
- coherent structures,
- high-variance trajectories,
- averaged modes.

2.2 Temporal Diffusion Parameter a

The file `compute_a.py` computes the temporal diffusion parameter a, which balances spatial and temporal diffusion scales. The temporal diffusion parameter a determines the relative strength of diffusion in time compared to space.

Let

$$\tau = T_f - T_0$$

denote the total time span, and let s_1, s_2 denote spatial domain lengths. Define

$$s_{\max} = \max(s_1, s_2).$$

A heuristic lower bound for a is given by

$$a_{\min} = \frac{\tau}{s_{\max}}.$$

This ensures that temporal diffusion is not negligible relative to spatial scales.

The final parameter is chosen as

$$a = \gamma \cdot a_{\min},$$

where γ is a user-defined multiplier.

2.3 Bandwidth Parameter ε

The file `epsilon_selection.py` determines the diffusion maps bandwidth parameter ε using a log-log slope heuristic.

For particle data

$$X = \{x_i\}_{i=1}^{N},$$

the kernel sum

$$S(\varepsilon) = \frac{1}{N^2} \sum_{i,j} K_{ij}(\varepsilon)$$

is computed using the Gaussian kernel

$$K_{ij}(\varepsilon) = \exp\left(-\frac{\|x_i - x_j\|^2}{4\varepsilon}\right).$$

Only pairwise distances satisfying

$$d_{ij} \leq \sqrt{5\varepsilon_{\max}}$$

are retained to reduce computational cost.

The optimal bandwidth is selected from the log-log slope

$$\frac{d \log S(\varepsilon)}{d \log \varepsilon},$$

which is smoothed using a Gaussian filter to reduce numerical noise.

The intrinsic dimension estimate is computed as

$$\hat{d} = 2 \times \text{slope}(\varepsilon_{\text{opt}}).$$

The selected bandwidth is saved in

`epsilon_opt.npy`,

and the heuristic figure is stored as

`epsilon_opt.png`.

3 Diffusion Maps, Spacetime Construction, and SEBA

This section implements the diffusion maps approximation, the spacetime diffusion operator using Strang splitting, and the Sparse Eigenbasis Approximation (SEBA) used in the inflated dynamic Laplacian framework of Section 6.1.

3.1 Diffusion Maps

The file `diffusion_maps.py` constructs the diffusion maps operator from particle trajectory data.

Nearest-neighbour distances are computed using a KDTree. For a point cloud

$$A = \{x_i\}_{i=1}^{N},$$

the nearest-neighbour distance is obtained from the second closest point:

$$d_i = \min_{j \neq i} \|x_i - x_j\|.$$

The mean squared nearest-neighbour distance is then

$$\left(\frac{1}{N} \sum_{i=1}^{N} d_i \right)^2.$$

For the diffusion maps matrix, a Gaussian kernel is used:

$$K_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{4\varepsilon} \right).$$

Only neighbours satisfying

$$\|x_i - x_j\| \leq \sqrt{5\varepsilon}$$

are retained to reduce computational and memory cost.

3.2 Temporal Laplacian

The temporal Laplacian is constructed from the discrete time vector

$$T_{\text{span}} = \{t_0, t_1, \ldots, t_n\}.$$

Using finite differences, the temporal Laplacian matrix approximates the second derivative operator

$$\frac{\partial^2}{\partial t^2}.$$

Boundary rows use one-sided differences, while interior rows use centered finite-difference approximations.

3.3 Spacetime Diffusion Operator

The file `spacetime.py` implements the spacetime diffusion operator using a Strang-splitting approximation.

For each time slice t_k, a spatial diffusion maps matrix

$$P_x^{(k)}$$

is constructed. These matrices are assembled into the block-diagonal operator

$$P_x = \text{blockdiag}\left(P_x^{(1)}, P_x^{(2)}, \ldots, P_x^{(T)}\right).$$

The temporal diffusion operator is generated from the temporal Laplacian:

$$P_t^{1/2} = \exp\left(\frac{\varepsilon}{2} L_t\right),$$

where L_t is the temporal Laplacian matrix.

The full spacetime diffusion operator is applied in product form:

$$P_a = P_t^{1/2} P_x P_t^{1/2}.$$

The function `matvec_Pa` applies this operator efficiently without explicitly forming the full dense matrix.

3.4 Sparse Eigenbasis Approximation (SEBA)

The file `seba.py` implements Sparse Eigenbasis Approximation (SEBA) to extract sparse coherent structures from eigenvectors.

Given an eigenvector matrix

$$V \in \mathbb{R}^{p \times r},$$

a QR decomposition is first applied:

$$V = QR.$$

The algorithm iteratively computes a sparse rotated basis using soft-thresholding:

$$s_i = \text{sign}(z_i) \max(|z_i| - \mu, 0),$$

where

$$\mu = \frac{0.99}{\sqrt{p}}.$$

After thresholding, a polar decomposition step updates the rotation matrix using singular value decomposition (SVD).

The iteration continues until convergence:

$$\|R_{\text{new}} - R\| < \text{tol}.$$

Finally, the sparse modes are normalized and sorted to produce coherent structure indicators.

4 Inflated Dynamic Laplacian Solver and Visualization

This section implements the inflated dynamic Laplacian solver together with the visualization routines used to analyse coherent structures in the switching double-gyre flow.

4.1 Inflated Dynamic Laplacian Solver

The file `solver.py` defines the class

`InflatedDynamicLaplacian`,

which computes spacetime coherent structures from trajectory data.

Given particle trajectories

$$\text{pts} \in \mathbb{R}^{2\times N\times T},$$

the solver determines the spatial diffusion scale ε_x. If no value is specified, it is estimated automatically from nearest-neighbour distances. The temporal diffusion scale is chosen from the timestep size Δt.

For total integration time

$$\tau = T_f - T_0,$$

the minimum temporal scaling parameter is

$$a_{\min} = \frac{\tau}{\max(L_x, L_y)},$$

and the final parameter is

$$a = a_{\text{factor}} \times a_{\min}.$$

The spacetime diffusion operator is constructed using the Strang-splitting form

$$P_a = P_t^{1/2} P_x P_t^{1/2}.$$

Eigenpairs of P_a are computed using the sparse eigensolver `eigs`.

The diffusion-map eigenvalues are converted into Laplacian eigenvalues using

$$\Lambda_k = \frac{\log(\nu_k)}{\varepsilon_x},$$

where ν_k are the dominant eigenvalues of the diffusion operator.

The eigenvectors are reshaped into spacetime form:

$$\text{evecs\_3d} \in \mathbb{R}^{N\times T\times K}.$$

4.2 Mode Classification

The solver separates modes into spatial and temporal categories.

For each eigenvector, slice-wise means are computed over time:

$$m_k(t) = \frac{1}{N} \sum_{i=1}^{N} v_k(i,t).$$

The temporal variance

$$\mathrm{Var}(m_k)$$

is used for classification, where low variance indicates spatial modes and high variance indicates temporal modes.

4.3 Visualization Routines

The file `plotting.py` provides visualization routines for analysing coherent structures in spacetime. The stream function

$$\psi(x_1, x_2, t) = -\sin\Big(\frac{\pi x_2}{2}\Big) \sin\big(\alpha x_1^2 + \beta x_1\big)$$

is visualized to show the changing gyre geometry. The code also generates eigenvalue spectra, 3D spacetime embeddings, snapshot grids at selected times, high-variance trajectory plots, and averaged spatial diffusion modes. A blue-white-red colormap is used throughout the visualizations to distinguish positive and negative coherent regions.

4.4 Main Execution Script

The file `run_section6_1.py` executes the complete pipeline for the switching double-gyre example.

The script:

- loads trajectory data from a MATLAB file,
- constructs spacetime arrays,
- initializes the inflated dynamic Laplacian solver,
- computes eigenvalues and eigenvectors,
- classifies coherent modes,
- generates all figures from Section 6.1.

The parameters used are:

$$\varepsilon = 0.0044, \qquad a_{\text{factor}} = 3.0$$

All generated figures are stored in the directory

`results_section6_1`.